Wiley
IET Software
Volume 2024, Article ID 1905538, 15 pages
https://doi.org/10.1049/sfw2/1905538

Research Article

An Observational Study on Flask Web Framework Questions on Stack Overflow (SO)

Luluh Albesher and Reem Alfayez

College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia

Correspondence should be addressed to Luluh Albesher; 443203817@student.ksu.edu.sa

Received 17 November 2023; Revised 16 September 2024; Accepted 6 November 2024

Academic Editor: Alessandro Marchetto

Web-based applications are popular in demand and usage. To facilitate the development of web-based applications, the software engineering community developed multiple web application frameworks, one of which is Flask. Flask is a popular web framework that allows developers to speed up and scale the development of web applications. A review of the software engineering literature revealed that the Stack Overflow (SO) website has proven its effectiveness in providing a better understanding of multiple subjects within the software engineering field. This study aims to analyze SO Flask-related questions to gain a better understanding of the stance of Flask on the website. We identified a set of 70,230 Flask-related questions that we further analyzed to estimate how the interest towards the framework evolved over time on the website. Afterward, we utilized the Latent Dirichlet Allocation (LDA) algorithm to identify Flask-related topics that are discussed within the set of the identified questions. Moreover, we leveraged a number of proxy measures to examine the difficulty and popularity of the identified topics. The study found that the interest towards Flask has been generally increasing on the website, with a peak in 2020 and drops in the following years. Moreover, Flask-related questions on SO revolve around 12 topics, where Application Programming Interface (API) can be considered the most popular topic and background tasks can be considered the most difficult one. Software engineering researchers, practitioners, educators, and Flask contributors may find this study useful in guiding their future Flask-related endeavors.

1. Introduction

Web-based applications have matured over the years and became an integral part of our daily routines [1, 2]. With the widespread usage of the Internet, these applications continue to be an essential part of a broad range of domains, such as entertainment, health, financial, education, and more [2, 3]. In comparison to traditional software applications, web-based applications are more susceptible to frequent changes in functionality, structure, and implementation; thus, they pose their own challenges [4]. Unfortunately, web-based applications are also susceptible to fail [4, 5]. Existing studies attribute web application failures to the time-consuming development process, developer's perception of the level of difficulty that is required to develop web applications, and web developers' struggle to cope with the frequent and several changes that are demanded to scale their web applications [4, 5]. With an aim to overcome challenges that reside within the development of web applications, the software engineering community has introduced several web frameworks to aid developers in building their applications efficiently and effectively by providing a set of tools and libraries that they can utilize [6, 7], one of which is Flask [8].

Flask is a popular Python web framework that provides developers with a flexible approach to building web-based applications by offering plenty of features and a collection of tools [1, 9]. The framework is known for its simplicity and ease of use, as it provides core functionalities while granting developers the ability to extend and add features as needed during the development process [1, 9–11]. The framework has been utilized to build several famous websites, such as LinkedIn and Pinterest [7]. Though the framework was officially released in April 2010 by Armin Ronacher [9], the framework is still ranked as one of the top 10 most widely used web frameworks in addition to being one of the top 20 most loved web frameworks, as evidenced by the Stack

Overflow (SO) 2022 survey [12], in which over 70,000 developers from over 180 countries participated. Moreover, the Python developers survey [13], which was a joint effort between the Python Software Foundation and JetBrains, revealed that Flask has been chosen as the most popular web framework for four consecutive years from 2018 to 2021.

A review of the software engineering literature highlighted the value of SO [14] in gaining a better understanding of many subjects within the software engineering field [15–27] due to the fact that the website is one of the most popular and largest technical online Question-and-Answer (Q&A) forums that provides developers with a platform to share and gain experiences by seeking peer answers to their developments questions in addition to answering other users questions [16, 17]. Specifically, analyzing questions posted on the website proved to be fruitful in understanding the interest towards a particular subject. In addition, utilizing topic modeling techniques, such as Latent Dirichlet Allocation (LDA) [28], on SO posts related to a particular subject can reveal topics that are asked about within the subject. Moreover, applying a number of proxy measures can help estimate the popularity and difficulty of these identified topics [16, 17, 19, 21, 24–27, 29].

Though many studies have utilized LDA on SO posts related to web development to understand real-world challenges faced by web developers [15, 29–31], no study has focused specifically on Flask. Therefore, aiming to attain deeper insights into Flask's stance on SO, we conducted an empirical study that is focused on Flask-related questions on the SO website. Having a deeper insight into Flask's stance on SO can offer numerous advantages. Flask practitioners can gain valuable insights into the challenges they might face when working with Flask, as well as within the broader scope of web development. Flask contributors and software engineering researchers can identify opportunities for improvement in both Flask and web development as a whole. Moreover, by revealing the issues encountered in Flask usage, software engineering educators can customize their curricula to effectively address these challenges and better prepare their students for real-world web development scenarios.

This empirical study began by acquiring a set of 70,230 SO Flask-related questions. We analyzed the set of questions to understand how the interest towards Flask evolved over time on the website. Then, we utilized LDA, a topic modeling algorithm, to identify Flask-related topics that are discussed within the set of the identified questions [28]. Afterward, we utilized a number of proxy measures to estimate the popularity and difficulty of the identified topics, and, lastly, we examined the monotonic correlation among popularity and difficulty proxy measures. This study's primary contributions can be summarized as follows:

1. Identifying and analyzing a set of 70,230 SO Flask-related questions.
2. Examining the interest towards Flask on SO over time.
3. Identifying the 12 main topics that SO users are asking about with regards to Flask.
4. Assessing the popularity and difficulty of the 12 identified Flask-related topics.

The rest of the paper is organized as follows: Background information is described in Section 2. The setup of the study is presented in Section 3. Section 4 presents the study's results, Section 5 discusses the study's results, and Section 6 summarizes the implications of the study. Section 7 discusses threats to the validity of the study and their corresponding mitigation strategies, once applicable. Section 8 summarizes an overview of previous, related work. Lastly, we conclude in Section 9.

2. Background

This section presents overviews of Flask, Stack Exchange network and SO, and the LDA algorithm. We note that some parts of Flask's overview are derived from Flask's official documentation and that the summarized overviews of the Stack Exchange network and SO are based on the Stack Exchange network websites meta.

2.1. Flask. Flask is a popular web framework that is written in Python. It was developed by Armin Ronacher of Pocoo, a global community of Python enthusiasts established in 2004 [7]. The framework was originally released as an April Fool's joke in 2010 and advertised itself as "the next generation Python micro web-framework" [9]. Despite the fact that the framework was released as an April Fool's joke, it caught users attention and became one of the most widely used frameworks for developing web applications [9].

Flask is primarily used as a back-end web framework [32]. The framework relies on the Werkzeug Web Server Gateway Interface (WSGI) toolkit and the Jinja2 template engine [9]. The Werkzeug WSGI toolkit is used as an interface between applications and servers to implement request and response objects, and Jinja is a template language that renders web pages [9]. As stated earlier, the framework is classified as a micro web-framework of a lightweight nature since it was originally designed to be scalable by allowing developers to add more functionalities as needed [9–11]. In other words, it is intended to keep the framework's core simple and extensible by allowing developers to incorporate external libraries, external extensions, or even creating their own extensions [6, 10]. This extensibility enables developers to have greater control over the development process [10].

As Flask depends on the Python language, the framework enables developers to create web applications quickly and efficiently without a steep learning curve [9, 10]. Due to the framework's simplicity, flexibility, and ease of use [1], many users have embraced the framework, in addition to many significant companies, such as Pinterest and LinkedIn [7].

2.2. Stack Exchange Network and SO. The Stack Exchange network [33] is a network of over 170 Q&A online websites that span over multiple fields. These websites share the same concept: seeking for help from other users and sharing experiences with others. Users can ask questions on a topic, answer other users' questions, or provide comments. Each question must include at least one tag, and it can have up to five tags. These tags are intended to index questions based on topics, aiming to facilitate the categorization and search of

questions [21]. A user can mark an answer that their question received as an accepted answer, indicating their satisfaction with the received answer [21]. In addition, users can also vote up or vote down other users' content (i.e., questions, answers, or comments) based on how they perceive the usefulness and appropriateness of the content. The websites on the network are self-managed through a reputation system, where users are rewarded and can earn reputation points based on their content. A user's reputation increases when other users vote up their content, and it decreases if the content is voted down.

The most popular Stack Exchange website is SO, an online platform that was launched in 2008. The website is intended to help both beginners and expert programers with their programing-related issues, such as an algorithm, encountered bugs, testing, or solving a programing challenge. As of Dec 07, 2022, SO has 19,330,674 million users, 23,147,406 million questions, 34,321,516 million answers, and 87,107,650 million comments. It was also estimated that the website receives over 200 million monthly visits, on average, and that over 150,000 questions, on average, are posted monthly on the website.

2.3. LDA. Topic modeling is an unsupervised machine-learning technique that aims to identify topics within a large corpus of text. The most commonly used algorithm for topic modeling is LDA [16, 19, 21, 24, 26, 27, 34], which was introduced in 2003 by Blei, Ng, and Jordan [28]. LDA has proven its effectiveness in molding topics and its superiority over other topic modeling algorithms, such as Probabilistic Latent Semantic Analysis (PLSA), in generating topics that are more interpretable and less prone to overfitting [24]. The algorithm has been widely and successfully utilized within the field of software engineering to extract topics that are discussed within a specific subject [16, 17, 19–21, 24–27, 35–37].

In the context of this study, the algorithm receives a set SO of questions as an input. Afterward, the algorithm clusters words in the set of inputted questions into a set of topics. The clustering is based on the frequency of words cooccurrence. The algorithm assigns each question a probability score for each identified topic. These assigned probabilities indicate the likelihood relevance of each question to each of the identified topics, where a topic that receives the highest probability score for a question is considered the most relevant topic to the question [16, 17, 21]. The algorithm also provides a list of the top 10 keywords for each identified topic as an output [19, 21].

One challenge that resides within the utilization of LDA is that the algorithm does not provide the optimal number of topics; instead, the number of topics must be provided as an input parameter [17, 19, 21, 23–25]. Identifying the optimal number of topics is particularly important as the quality of the generated topics is highly dependable on the inputted number of topics, as inputting a small number of topics may lead to overlapping topics, and inputting a large number of topics may produce extremely fine-grained topics [17, 19, 21, 23, 28]. Therefore, to overcome this challenge, one needs to conduct multiple experiment runs in which they vary the number of topics in each run and record the resulting coherence score (i.e., an indicator of the resulting model quality) [17, 19, 21, 23–25]. A higher coherence score generally suggests better model quality, though it does not provide an absolute guarantee. Once the experiment runs have been finalized, the model that resulted in the highest coherence score can be considered as the optimal model, and hence, its number of topics can also be considered optimal [17, 19, 21, 23–25].

Another challenge that is imposed by the utilization of LDA is the naming of topics, as the algorithm does not provide names (i.e., human-readable topics) for the identified topic [16]. If one wishes to name the identified set of topics, they have to follow a manual approach to accomplish that, as there is no automated approach that can achieve such a goal [16, 17]. Therefore, one needs to utilize a manual approach to name these topics [16], such as thematic analysis [38] or open card sort [39]. Rather, those who are involved within the naming of topics need to develop criteria by which the topics will be named, and, subsequently, name the topics. Hence, one can review the top 10 keywords that are outputted by the algorithm for each identified topic and a number of the most relevant questions of each topic to assign it a name that best represents the keywords and questions of each identified topic [19, 21]. Afterward, one can select random questions from each topic to verify the suitability of the assigned name [17, 19].

3. Study Setup

This section presents the setup of the study, including the study's research questions and dataset.

3.1. Research Questions. The study aims to answer three research questions that we have derived to attain the primary objective of this study: gaining a better understanding of Flask's stance on SO.

3.1.1. Interest. Though Flask was officially released over a decade ago, the popularity of the framework did not fade out [9]. Not only tens of thousands of web applications were developed using Flask [40], but as stated in Section 1, the framework has also been continuously ranked as the most popular web framework since 2018–2021 by the Python developers survey. Moreover, the 2022 SO survey revealed that the framework was among the top 10 most widely utilized web frameworks. Given that the primary objective of this study is gaining a better understanding of the stance of Flask on SO, we aim to understand the interest towards the framework on the website and how it evolves over time. This leads to this study's first research question being: *RQ1: How does the interest towards Flask web framework on SO evolve over time?*

3.1.2. Topics. The demand and usage of web-based application development is continuously growing [1]. Many users have expressed interest in utilizing the Flask web framework in their web applications [9, 40]. Despite the framework's aim to ease the process of web application development

[1], web developers may still encounter some barriers within the development of web applications [4, 5]. As this study aims to gain a better understanding of the stance of Flask on SO, it is only imperative to seek an understanding of Flask-related topics that are discussed by SO users. Additionally, examining how the set of identified topics evolves over time can provide insights into how Flask users' interests in different topics shift. For example, it is expected that the number of questions related to the deployment of Flask applications will be high in the early years and decrease as the framework becomes more established. Therefore, this study's second research question is: *RQ2: What Flask-related topics are discussed on SO and how they evolve over time?*

3.1.3. Topic Characteristics. Though the second research question aims to provide clarity on Flask-related topics that are discussed by SO users, the question fails to provide an understanding of the level of popularity and difficulty of the identified topics. Knowing which Flask-related topics are more popular can be valuable in determining which Flask-related topics are of more interest to the website's users [17, 19, 21, 24, 25, 29]. Moreover, having an understanding of which Flask-related topics are considered to be more difficult can aid in providing clarity on topics that are currently lacking solutions and requiring more attention to be resolved [17, 19, 21, 24, 25, 29]. Furthermore, having an understanding of the potential monotonic correlation between popularity and difficulty can provide a more holistic understanding regarding the relation among popularity and difficulty proxy measures [19, 24, 25]. Hence, this study's third research question is: *RQ3: What are the characteristics of the identified topics in terms of popularity and difficulty, and how are popularity and difficulty related?*

3.2. Dataset. The first step to answering this study's research questions is to acquire a dataset that contains SO Flask-related questions. As a result, we utilized the publicly available Stack Exchange Data Dump [41]. The dump contains anonymized data of all users' contributed content on the Stack Exchange network in XML files. We particularly utilized the SO dump that contains data up until Dec 07, 2022. To identify Flask-related questions, we utilized a search method that encompasses both tag-based and content-based filterings, following prior related studies [19, 21, 22]. Mainly, we retrieved any question that contains the term "flask" in its title, body, or tag, ignoring letter case sensitivity. This step resulted in the retrieval of a total of 70,237 questions. To verify that the retrieved questions are Flask-related, we followed the recommendation of [42] and randomly selected a representative random sample that consists of 383 questions based on a 95% confidence level and a 5% margin of error. Subsequently, both authors independently reviewed that random sample to determine the questions relevance to the framework. Afterward, in a joint meeting, the two authors compared their decisions. The meeting was initiated by calculating Cohen's Kappa coefficient to measure the interrater agreement [42]. The resulting coefficient was equal to 1.0, indicating perfect agreement between the two authors [43]. The review resulted in the observation that some questions that contain the term "flask" reference a container that is typically used in chemistry laboratory settings instead of referencing the framework. Therefore, we compiled a list of terms and searched for any question that contains any of the terms in its title, body, or tag. The list of terms is "chemi, lab, and ingredient_table." We note that the term "chemi" was included as it can retrieve multiple forms of the word "chemistry," such as "chemical," "chemistry," and "alchemist." This step retrieved 70,230 questions that both authors independently reviewed to determine their relevance to the framework. A subsequent joint meeting was held, and the calculation of Cohen's Kappa coefficient resulted in 1.0, indicating perfect agreement between the two authors. Both authors decided to omit seven questions, as they were deemed not to be related to the framework. Afterward, we selected a second random representative sample that consists of 383 questions, based on a 95% confidence level and a 5% margin of error. We independently reviewed the obtained sample. Then, we jointly compared the results and calculated the Cohen's Kappa coefficient, which indicated total agreement. The review deemed all retrieved questions to be Flask-related. Consequently, all of the retrieved 70,230 questions were utilized to answer this study's research questions.

4. Results

This section summarizes the findings of this study. The findings are derived from the set of 70,230 SO Flask-related questions.

4.1. RQ1: How Does the Interest Towards Flask Web Framework on SO Evolve Over Time?

4.1.1. Approach. To gain a better understanding of the interest towards Flask on SO, we utilized two proxy measures that have also been utilized by previous related studies [17, 25]. Particularly, we calculated the total number of Flask-related questions posted yearly and the yearly total number of new unique users who posted a Flask-related question [17, 25]. The yearly total number of Flask-related questions can provide a general indication of the interest towards Flask on SO [17, 25]. The number of unique users who posted a Flask-related question can provide clarity on whether Flask-related questions are posted by few, frequent users or a large number of users [17, 25]. The count of yearly unique users was calculated by counting the number of users who posted their first Flask-related question in each year. In other words, we sorted the Flask-related questions asked by each user in chronological increasing order and only counted each user once in the first year they posted a Flask-related question.

4.1.2. Results. Frequency of questions: The yearly total number of Flask-related questions is depicted in Figure 1. As the figure illustrates, Flask-related questions have been present on the website since the framework's early years. Though the number of questions was increasing through the years, with a peak in 2020, it dropped in 2021 and 2022.

Number of unique users: We found that a total of 43,896 unique users had posted a Flask-related question since the

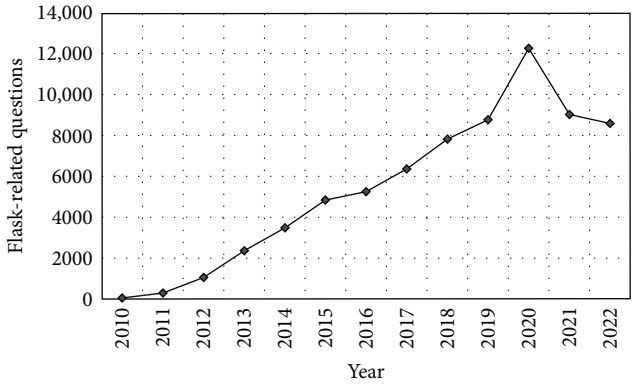

FIGURE 1: The distribution of Flask-related questions per year.

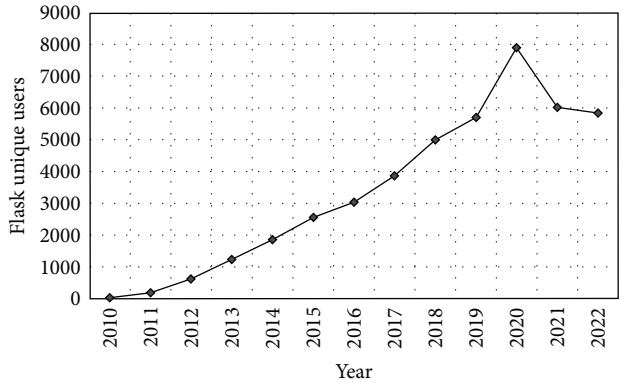

FIGURE 2: The distribution of unique users who post Flask-related questions per year.

framework's official release in 2010 until 2022. Figure 2 depicts the yearly breakdown of Flask unique users. As Figure 2 illustrates, the trend of the yearly number of unique users is similar to the trend of the yearly total number of Flask-related questions. The yearly number of unique users who posted a Flask-related question is generally increasing, reaching a peak in 2020 but showing a decline starting in 2021.

We furthered our analysis by calculating the number of Flask-related questions each user had asked, and we found that out of the 43,896 SO unique users who posted Flask-related questions, there were around 46.83% who only asked one question, 8.58% who asked two questions, 5.39% who asked between three and five questions, 1.25% who asked between six and 10 questions, and only 0.44% who asked more than 10 questions. The maximum number of questions asked by one user is 103, with a mean of 1.59 questions per user and a median of 1.

4.2. RQ2: What Flask-Related Topics Are Discussed on SO and How They Evolve Over Time?

4.2.1. Approach. Aiming to identify topics that are discussed within the set of 70,230 SO Flask-related questions, we utilized LDA to model the set of acquired questions into topics. Specifically, we followed the approach depicted in Figure 3. We initially preprocessed the dataset to reduce any noise that might influence the resulting LDA model [23]. The preprocessing step began by removing all code snippets in addition to all HTML tags and URLs, such as <*p*> and <*a*> [19, 21, 24, 25, 29]. Then, we tokenized each question into a list of words [17]. Then, the NLTK stop words [44] were used to remove common words, such as "the," "is," and "are" [19, 24, 25, 29]. Afterward, spaCY Lemmatizer [45] was utilized to reduce words to their base form, such as transforming "developing," "developers," and "development" to "develop" [25]. After completing the preprocessing steps, Gensim was used to construct a bigram model, as bigram models have demonstrated their ability in improving the quality of textual processing [19, 21, 29]. Following that, we utilized the MALLET [46] implementation of LDA to identify topics that are discussed within the acquired set of Flask-related questions. We decided to use the MALLET implementation as it produces better-quality topics in comparison to other library implementations [19, 21, 24, 29].

As mentioned previously in Section 2, the LDA does not identify the optimal number of topics, and it instead requires the number of topics as an input [17, 19, 21, 23–25]. Therefore, we conducted multiple preliminary experiments, following the approach of previous related studies [19, 21, 24, 25, 29]. We initiated the experiments by setting the number of topics to two then we incremented the number of topics by an increment of two in each subsequent experiment run. In addition, we recorded the coherence score of each run. The experiments were concluded when we reached 50 topics, as the coherence score decreased drastically. After we concluded the experiments, we compared the resulting coherence scores and found that the highest coherence score equals 0.59, suggesting that Flask-related questions can be categorized into 12 topics.

To name the set of the 12 identified Flask-related topics, we performed a thematic analysis [38]. Particularly, we employed an inductive approach in which there is no predefined set of topic names. Instead, the name of each topic is derived during the analysis. The two authors reviewed the resulting top 10 keywords in addition to the top 50 questions (i.e., questions with the 50 highest probabilities of belonging to a topic) of each identified topic and assigned each topic a name. To confirm the suitability of assigned names, we selected and reviewed a random representative sample from each topic's questions [19, 21, 24, 29], and the review confirmed the suitability of all assigned names.

After identifying and naming the set of Flask-related topics discussed on SO, we calculated the total number of questions posted each year for each identified topic. We then plotted the results in a trend line graph to visualize the changes and trends in these topics over time.

4.2.2. Results. The aforementioned approach resulted in a total of 12 topics, which are summarized in Table 1, along with their associated top 10 keywords and frequency. The topics are summarized below, ordered by decreasing frequency.

Database: A total of 8074 Flask-related questions fall under the database topic. Questions in this topic discuss

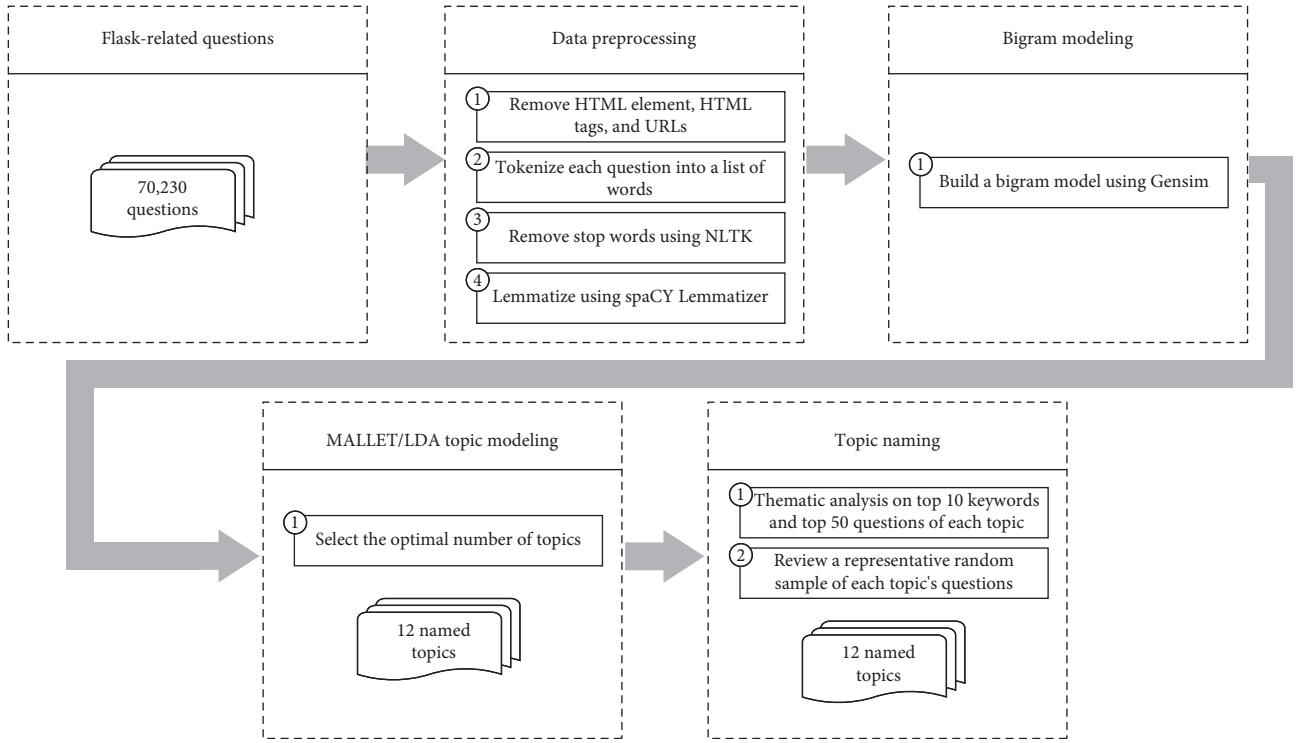

FIGURE 3: An overview of topic identification approach.

TABLE 1: Flask-related topics with their associated top 10 keywords and frequency.

Topic name	Keywords	Frequency
Database	database, table, model, create, query, class, sqlalchemy, column, add, update	8074
Form	form, template, display, view, field, button, click, render, input, submit	7735
Deployment	run, server, deploy, service, container, local, connect, host, port, nginx	7425
File handling	file, image, script, load, folder, upload, directory, structure, main, save	6881
Setup	error, import, module, project, version, install, command, package, site_package, exception	6324
Passing variables	function, return, object, pass, variable, output, code, print, list, result	5958
Client–server	request, server, datum, send, post, message, code, client, response, receive	5451
Background tasks	time, process, start, connection, task, update, thread, execute, worker, run	5253
API	api, route, url, access, method, endpoint, call, backend, implement, rest	4764
Authentication and authorization	user, create, set, login, session, check, store, code, redirect, link	4460
Testing and logging	work, code, problem, log, issue, test, change, edit, wrong, solve	4010
Best practices	question, solution, simple, answer, understand, point, thing, framework, read, figure	3895

Abbreviation: API, Application Programming Interface.

database connections, creating models and tables, altering tables, and persisting data to the database. Flask-SQLAlchemy [47] (i.e., a Flask extension that adds support for SQLAlchemy) and Flask-Migrate [48] (i.e., a Flask extension that handles SQLAlchemy database migration) are frequently referenced by SO users in this topic. Examples of questions in this topic are "*SQLAlchemy in Flask—How to auto update the same column in another table*," "*How can I join two tables with different number of rows in Flask SQLAlchemy?*," and "*How to use flask-migrate to migrate multi database with different table*."

Form: In the context of web development, a form refers to a web element that is used to collect users' input [10]. A total of 7735 questions were found to be relating to form in the set of the identified Flask-related questions. Questions in this topic inquire about several form aspects, including how to create a form, such as "*Use FlaskForm and wtforms to create form with variable number of fields*." Users also ask about how to validate form data, how to display form fields, how to process form data, and how to render forms in templates. Moreover, questions in this topic include questions of both the Flask WTForm [49] (i.e., a flexible forms validation

and rendering library for Python) and the regular web forms [10]. "*How to manually create a form and then validate it in Flask wtform?*," "*Process Data within Flask WTForms Submission*," and "*Flask Form doesn't render in extended template*" are examples of such questions.

Deployment: Deployment refers to the process of making an application available to its end users [50]. The deployment of the developed web applications was the central topic of 7425 Flask-related questions. Questions in this topic include configuring web servers (e.g., Apache [51] and Nginx [52]), deploying to cloud services (e.g., Heroku [53] and AWS [54]), and deploying Flask applications using containers (e.g., Docker). Examples of such questions are "*How to configure Apache under WAMPSERVER* (*Windows*) *to run a Flask application?*," "*How to copy Python virtual environment flask project to another server and deploy the application in that server?*," "*How to deploy Flask application with Nginx and uWSGI?*," and "*Deploying a minimal flask app in docker —server connection issues*."

File handling: File handling refers to storing, organizing, and accessing files properly [55]. A total of 6881 Flask-related questions were identified to be relating to the file handling topic. Questioners inquired about how to upload files to servers, such as "*Error when uploading a zipfile to a python app hosted on Heroku*." Questions also include asking how one can download files in Flask, such as "*How to download a file from Flask with send_from_directory?*," and how one can handle large files, zip files, and the loading of a file, as exemplified by "*Flask—Delete zipfile after download*." In addition, SO users asked about how one can correctly structure or write a file path to access or store a file, such as "*How to deal with file paths in Flask*."

Setup: Setup refers to the process of preparing the development environment for use [9]. Flask setup was the main topic of a total of 6324 Flask-related questions. Questions in this topic range from simple questions, such as asking about how to install Flask in a virtual environment; how to import packages in Flask projects; and how to update Flask versions, to more advanced questions, such as how to configure Flask with other development environments; how to manage Flask dependencies in package manager tools (e.g., Conda and Pipenv); and how to execute custom command lines using Flask. "*Why I can't import Flask?*," "*ModuleNotFoundError when importing package that is installed in conda environment*," "*Flask runs out of virtual environment*," and "*How to run Flask app by flask run or custom command without set FLASK_APP manually*" are examples of questions in this topic.

Passing variables: Passing variables refers to the process of sending data from one part of an application to another [11]. Passing variables was the central topic of 5958 Flask-related questions. Questions in this topic include inquiries about passing variables to view functions (i.e., functions that handle the application URLs [10, 11]), passing variables to Flask templates, and passing variables between JavaScript code and Flask templates. "*How to pass an array of formatted sentences to a Flask template?*," "*Pass parameter to view function from Jinja templates in Flask*," and "*How to pass JavaScript list of directory variables to Python Flask*" are examples of such questions. In addition, SO users inquired about how to pass variables between different routes (i.e., routes are used to bind a web page URL to a particular function) and how to pass the parameters of a route, such as "*How to Pass Dictionary between two different routes in Flask?*" and "*How to pass password in flask route*."

Client–server: Client–server refers to the data exchange between a client (i.e., requester for resources) and a server (i.e., provider of resources) [56]. The client–server topic was found to be the focus of 5451 of the identified Flask-related questions. Questions in this topic include inquiries regarding how to create a request, fetch request data, write response headers, and handle server responses. "*How can I create a request and response with JQuery Ajax and Flask*," "*How to fetch request data from flask post request*," and "*How do I set response headers in Flask?*" are examples of such questions. Moreover, users also sought assistance with issues that they encountered with servers' responses and clients' requests, such as "*Flask suddenly not receiving request from React client*" and "*Python Flask server getting 'code 400' errors (POST request sent from Telegram-webhook)*."

Background tasks: Background tasks are the processes that run independently without interrupting the main application [57], and the topic includes 5253 questions. SO users discussed several aspects of background tasks in this topic. Questions include asking about how to execute long-running tasks as standalone processes and how to run threads in the context of Flask, such as "*Flask and long running tasks*" and "*Run Flask single threaded*." Users also asked questions related to Celery [58] (i.e., a Flask task queue that can be used for background tasks), including questions about Celery message, Celery task, Celery task queue, and Celery worker. "*Celery tasks run twice in the background once Flask application is run, why?*" and "*Celery workers stop execution unexpectedly*" are examples of such questions. In addition, questions discussed issues related to message brokers (i.e., intermediary that facilitate message exchange between an application and Celery), including questions about Redis [59] and RabbitMQ [60] message brokers. "*Celery + Redis how to run tasks based on lowest timestamp with same priority*" and "*How to make celery consumes queue separately on each machines from one rabbitmq?*" are examples of these questions.

Application Programming Interface (API): API is a set of defined rules that facilitate the communication between different applications [61]. A total of 4764 Flask-related questions were identified to be related to the API topic. Questions in this topic include inquiries regarding how one can effectively design and develop an API in Flask, such as "*How to create Flask API without a model*" and "*How to add a new parameter to a Python API created using Flask*." Questions in this topic also include questions about how one can utilize available APIs, deal with the nuances of the varying versions of these APIs, and how one can handle API endpoints (i.e., a point at which an API connects with the software application [61]). "*Why use Flask open_resource*" "*Support multiple API versions in flask*," and "*How to handle multiple 'GET' endpoints in a Flask-Restplus response class*" are examples of such questions.

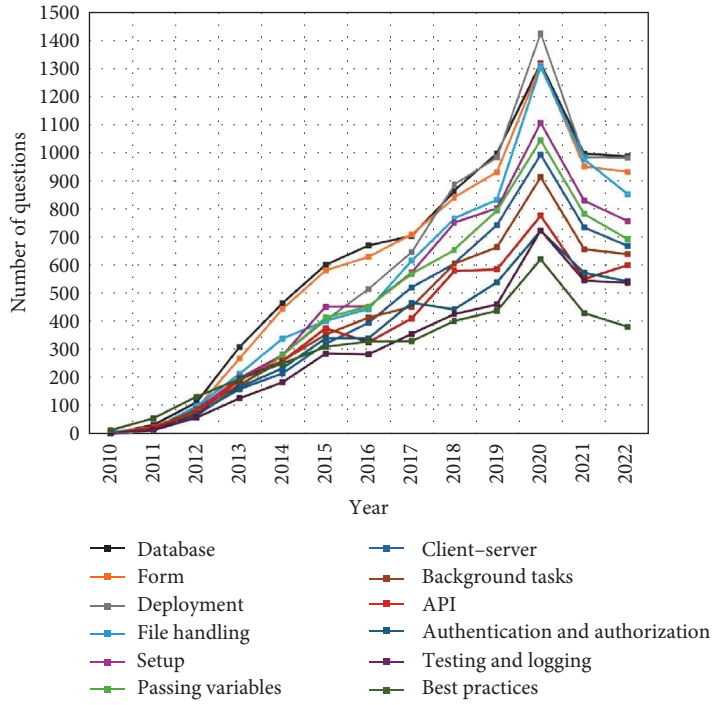

FIGURE 4: Trend line of identified topics.

Authentication and authorization: Authentication refers to validating credentials, and authorization is concerned with appropriately granting permissions to resources [62]. A total of 4460 Flask-related questions were found to be related to the authentication and authorization topic. Questions in this topic include asking about how to effectively implement users authentication mechanisms, such as "*How do you implement token authentication in Flask?*." Users also asked about how to setup an authorization mechanism that determines access levels and privileges for different user levels, such as "*Setting Authorization header—Flask app using Python Requests and JWT*." SO users also ask about how to create secure password hashing and how to utilize the Flask-login [63] extension (i.e., a Flask extension that provides user session management), such as "*Removing Session cookie in Flask & Flask-Login*." Other questions that were discussed in this topic pertain to problems that arise when integrating third-party authentication services, such as OAuth, LDAP, and OpenID, for example, "*Authenticate user with a specific hosted domain (hd) in Flask with Oauth2*." Moreover, questioners in this topic seek to find a plausible explanation of an encountered error that is related to authentication and authorization and how to fix it, such as "*How Do I Fix Errors Arising from working with Python sessions and flask_session*."

Testing and logging: Testing refers to validating the correctness level of an application's functionality while logging records of various types of information about an application's execution [10]. A total of 4010 Flask-related questions were identified as being related to the testing and logging topic. Questions in this topic include inquiries about how to effectively set up logging configurations and test environments, such as "*Flask properly configure logging*" and "*Choosing test config and settings in python flask project when running py.test*." In addition, questions include asking about how to output the results of logging into a file, such as "*How to log to a new file in every route handler function?*." Users also ask about how to write unit tests for specific features and how to debug errors, such as "*Log messages not printed by Flask in my custom Flask extension*," "*Login test failing in Python Flask unit test*," and "*How to unittest a function that use app.logger of Flask*."

Best practices: A total of 3895 Flask-related questions were identified as being related to the topic of best practices. Questions in this topic seek to know what are the best practices in Flask to conduct a variety of tasks. Questioners are aiming to benefit from other SO users' experiences regarding best practices, and they seek to gain insight from SO users based on their previous experiences with other alternatives, as exemplified by "*What is the best practice to assign a class to an HTML element based on a back end variable using flask and JINJA?*" and "*Flask-login alternative for beginner*." In addition, questions in this topic can also include inquiries that aim to understand why a particular solution is considered a best practice within the framework, where "*What is the point of initializing extensions with the flask instance in Flask?*" is an example of such questions.

Moreover, Figure 4 represents the trends of the Flask-related topics over the examined years. As Figure 4 depicts, the trends of these topics are similar; all of these topics generally followed an increasing trend until peaking in 2020, followed by a decline in the subsequent years. Both the

database and form topics consistently ranked among the top two in most of the examined years, while best practices and testing and logging remained among the least discussed topics.

4.3. RQ3: What Are the Characteristics of the Identified Topics in Terms of Popularity and Difficulty, and How Are Popularity and Difficulty Related?

4.3.1. Approach. To estimate the popularity and difficulty of each identified Flask-related topic, we utilized several proxy measures that have been adopted by previous related studies [19, 21, 24, 25, 29].

Particularly, to measure the popularity of each identified topic, we calculated the average number of views and the average number of scores its questions received. The average number of views can serve as an indicator of the interest towards a Flask-related topic from both SO users and visitors [17]. Measuring the interest of visitors is particularly important as SO has more visitors than registered users [64]. Furthermore, the score of a question results from summing the upvotes and downvotes the question received from SO users, which can indicate the perception of registered users towards the question [21]. Moreover, to rank the topics based on their popularity, we calculated a single measure that combines the two previous measures. Specifically, we utilized the approach of [24] and normalized the two popularity proxy measures by dividing the value of each measure by the average of its values across all 12 identified topics. Subsequently, for each topic, we computed the average of the two normalized popularity proxy measures to create a single measure that we used to rank the topics based on their popularity. Particularly, we used the following formulas:

$$\text{View}N_i = \frac{\text{View}_i \times 12}{\sum_{j=1}^{12} \text{View}_j}$$

$$\text{Score}N_i = \frac{\text{Score}_i \times 12}{\sum_{j=1}^{12} \text{Score}_j}$$

$$\text{Fused}P_i = \frac{\text{View}N_i + \text{Score}N_i}{2}$$

where i denotes a specific topic.

Moreover, to estimate the difficulty of a topic, we relied on two proxy measures. Mainly, we calculated the percentage of questions that did not receive an accepted answer and the median time elapsed in seconds for questions to receive an accepted answer for each identified topic. SO users who posted a question can indicate an answer that they received as an accepted if they find the answer to be satisfactory in addressing their question, and a lack of an accepted answer may indicate that the questioner is still struggling with their question [19, 22]. We have also calculated the median time elapsed to receive an accepted answer, as the success of a crowd-sourced platform, such as SO, depends highly on its users ability to provide timely correct answers [17, 19, 20, 22]. We utilized the median to avoid any bias that might result in the mean from long-latency answers, as the majority of SO questions are answered within the first hour of a question's posting [17, 19, 20]. Similar to the approach followed to evaluate the popularity of topics, we derived a single measure that combines the two difficulty proxy measures. Specifically, we divided each measure's value by its respective average across all 12 identified topics. Then, we computed the average of the two normalized difficulty proxy measures, following the approach of Uddin et al. [24]. Particularly, we used the following formulas:

$$\%W/o \text{ accepted answer}N_i = \frac{\%W/o \text{ accepted answer}_i \times 12}{\sum_{j=1}^{12} \%W/o \text{ accepted answer}_j}$$

$$\text{Median time (s)}N_i = \frac{\text{Median time (s)}_i \times 12}{\sum_{j=1}^{12} \text{Median time (s)}_j}$$

$$\text{Fused}D_i = \frac{\%W/o \text{ accepted answer}N_i + \text{Median time (s)}N_i}{2}$$

where i denotes a specific topic.

Lastly, we used the Kendall correlation test to assess the relationships between each topic's popularity and difficulty proxy measure [24, 65]. The test was utilized due to its ability to produce more reliable results and tolerance to outliers [24, 65].

4.3.2. Results. The statistics on the popularity of each identified Flask-related topic are presented in Table 2, where column "Fused*P*" presents the calculated fuzed popularity measure, column "AVG(views)" presents an average number of views, and column "AVG(scores)" presents an average number of scores. As illustrated in the table, the topic of API has the highest Fused*P*, and therefore, it can be considered the most popular topic. The topic of best practices has the second highest Fused*P*, followed by setup, client–server, passing variables, background tasks and deployment (tied), database, file handling, authentication and authorization, testing and logging, and form. Based on the aforementioned results, the topic of API can be considered the most popular Flask-related topic, followed by the best practices topic.

Table 3 presents the statistics on identified Flask-related topics' difficulty, where column "Fused*D*" presents fuzed difficulty measure, column "% *W/o* accepted answer" presents percentages of questions without an accepted answer, and column "Median time (s)" presents median time elapsed to receive an accepted answer in seconds. As presented in the table, the topic of background tasks has the highest Fused*D*, followed by deployment, API, testing and logging, database, authentication and authorization, file handling, client–server, setup, form, best practices (tied), and passing variables.

Therefore, it can be concluded that the topic of background tasks is the most difficult Flask-related topic, followed by the deployment topic.

Table 2: Popularity of Flask topics.

Topic name	Fused*P*	AVG(views)	AVG(scores)
Database	0.94	2155.57	1.83
Form	0.82	2168.27	1.38
Deployment	0.95	2211.92	1.85
File handling	0.93	2363.32	1.64
Setup	1.16	3144.95	1.91
Passing variables	0.96	2550.64	1.61
Client–server	1.04	2729.87	1.78
Background tasks	0.95	1898.57	2.10
API	1.25	2710.47	2.58
Authentication and authorization	0.87	2143	1.60
Testing and logging	0.83	1882.08	1.65
Best practices	1.23	2754.31	2.45

Abbreviation: API, Application Programming Interface.

Table 3: Difficulty of Flask topics.

Topic name	Fused*D*	% *W/o* accepted answer	Median time (s)
Database	1.05	54.43	9644.88
Form	0.78	53.65	5278.07
Deployment	1.29	62.75	12,423.89
File handling	0.85	58.23	5833.39
Setup	0.81	59.52	5032.80
Passing variables	0.61	51.68	2801.16
Client–server	0.84	59.51	5438.76
Background tasks	1.72	64.63	19,263.02
API	1.13	56.19	10,735.32
Authentication and authorization	1.01	56.66	8701.88
Testing and logging	1.08	61.80	9091.22
Best practices	0.78	54.22	5147.86

Abbreviation: API, Application Programming Interface.

Table 4: Correlation coefficients of popularity and difficulty proxy measures.

	View	Score
% *W/o* accepted answer	0.21	0.12
Time to accepted answer	0.03	0.08

The examination of the monotonic correlation among the popularity and difficulty proxy measures using the Kendall rank correlation test resulted in statistically significant correlations in all cases (i.e., all p-value instances < 0.01). The correlation coefficients are presented in Table 4, and it can be observed that though all obtained coefficients are positive, they are pretty small (i.e., less than or equal to 0.2), suggesting negligible correlations [66, 67].

5. Discussion

The results revealed that interest in Flask on SO has been steadily increasing since the early years of the framework until it peaked in 2020 to drop after in the 2 following years. While the number of questions has decreased, this trend might be influenced by a potential decline in SO's popularity, with developers possibly turning to alternative sources, such as ChatGPT, for their inquiries.

The results revealed that Flask-related questions revolve around 12 topics. The appearance of some of these topics was expected as they are considered to be typically challenging within software engineering development in general, such as authentication and authorization [23]. In addition, other topics were also expected to be included within Flask-related questions, such as database, deployment, and file handling, as these topics were identified to be challenging within web development in general by previous studies [15, 31]. However, other topics were not expected to appear, such as setup, as Flask's official documentation contains detailed guidelines and tutorials that aim to ease the process of setup. The variety of Flask-related questions topics on SO may be attributed to the diverse experiences of its users. While one question might come from a beginner who has just started exploring programing, another could be asked by an experienced developer with a long career in software development.

The results also revealed that the topics of API, best practices, and setup are among the most popular topics. As mentioned previously, this need for guidance in setting up the framework and following best practices was a bit of a surprise based on the assumption that users would refer to the official Flask documentation that contains detailed tutorials on these topics. The study also found that the topic of background tasks can be considered to be the most difficult topic, as it has the highest rate of questions without an accepted answer and requires the longest median time to receive an accepted answer. Finding that the topic of background tasks to be the most difficult topic is consistent with the finding of a previous, related work that noted that background tasks can be considered one of the most challenging topics in software engineering development [23]. Moreover, the examination of the monotonic correlation between each popularity and difficulty proxy measure resulted in very low positive coefficients, indicating a negligible correlation between popularity and difficulty.

6. Implications

SO has continuously proven its capability to provide valuable insights into the stance of a given subject within the software engineering field [16, 17]. Having such an insight can be beneficial to those who seek to gain a better understanding of a particular subject [16, 19, 20, 23, 68]. This study identifies and analyzes Flask-related questions on SO. This section summarizes how software engineering researchers, practitioners, educators, and Flask contributors can benefit from the findings of this study.

Software engineering researchers can leverage the findings of this study to guide their future research efforts focused on Flask. For example, the study identified background tasks as one of the most challenging topics, with many questions remaining without an accepted answer. Researchers can delve into the specific difficulties users face, such as executing long-running tasks as standalone processes or managing threads within the Flask context. By pinpointing these challenges, researchers can develop new solutions to address them or enhance existing solutions to better address the issues users encounter when managing background tasks in Flask applications and filling any gaps in current practices.

Software engineering practitioners can find the study and its findings beneficial in understanding Flask-related topics that are discussed on SO and the characteristics of these topics. Having prior knowledge regarding the challenges that might be encountered when using Flask can help practitioners in anticipating these issues to avoid them or be proactive about them. For example, practitioners can expect to face some difficulties when developing background tasks, and they can seek solutions to overcome these hurdles or find other alternatives. Practitioners can also realize that these issues are actually common among Flask developers, and hence, they can utilize SO to learn from other practitioners' experiences when having to deal with similar challenges. Another potential benefit is that practitioners can review questions that are related to a particular topic to learn about the techniques and trends within the topics, such as passing variables.

Software engineering educators can leverage the study and its findings to improve their web development curricula. For example, software educators may consider focusing on Flask-related topics that SO users are facing issues with, such as background tasks. The inclusion of such a topic may aid students in understanding background tasks, what they entail, and the common challenges associated with them. Educators can include projects and homework assignments on background tasks involving the use of Celery that cover Celery messages, tasks, task queues, and workers. Projects can also include working with message brokers like Redis and RabbitMQ. By integrating these real-world challenges and tools into their curriculum, educators can better prepare students for the complexities of web development with Flask and equip students with practical skills and experience to address common issues.

Flask contributors can utilize the results of this study to identify opportunities for improvement within the framework. For example, they can pinpoint challenges that the framework users are facing and eliminate or mitigate such challenges. One example is that the study found that both the topics of background tasks and deployment to be among the most difficult topics, indicating that these topics have higher rates of questions without an accepted answer and longer elapsed time to receive an accepted answer. Such findings can imply that Flask contributors may consider focusing on the exact challenges that the users are facing to eliminate or mitigate them. For instance, contributors can improve the experience of using Celery with Flask by identifying common errors and pitfalls then creating comprehensive, user-friendly documentation and developing step-by-step tutorials and guides.

7. Threats to Validity

This section presents potential threats to the external, construct, and reliability validities of the study and their corresponding mitigation strategies, once applicable.

7.1. External Validity. External validity refers to the extent to which a study's findings can be generalized and applied to other contexts beyond the scope of the study [69–71]. This empirical study's primary aim is to gain a better insight into the stance of Flask questions on SO. Although this empirical investigation revealed many findings on the stance of Flask on SO, we cannot claim the generalizability of these findings for several reasons. First, this study solely focused on SO, and therefore, there is a possibility that other Flask-related topics might have been discussed on other platforms. Second, though we strove to include all Flask-related questions in this study through the utilization of both tag-base and content-based filterings, there is a potential that SO users might have utilized other terms to reference Flask, and hence, we failed to include these questions. Therefore, the findings of this study should only be interpreted within the context of its boundaries and

may not be generalizable to questions that were not identified using the described search method nor on other websites.

7.2. Construct Validity. The construct validity of a study is concerned with the extent to which the utilized scales, metrics, and instruments actually measure what they are intended to measure [69–71]. One potential threat to this study's construct validity is the measurement of interest towards Flask and the popularity and difficulty of the identified Flask-related topics, as the results were solely derived from a number of proxy measures. To mitigate this threat, we only relied on known proxy measures that have been commonly utilized in related, previous work [17, 19, 21, 24–27, 29, 68, 72]. We note that these proxy measures are not perfect and have some limitations. For example, a drop in the number of yearly asked questions may be due to the availability of answers in previously asked questions, eliminating the need for users to ask again. Additionally, a question might not receive an accepted answer, but it may be adequately answered in a comment. Therefore, the study's findings should be interpreted within the context of the utilized proxy measures, keeping in mind their various limitations.

Another potential threat to the construct validity of this study is the usage of statistical measures and tests. In addition to the utilization of the common statistical measures (e.g., average and percentage) to convey the study's findings, we have only used the Cohen's Kappa coefficient to measure interrater agreement between the two authors and the Kendall correlation test to assess the relationship between difficulty and popularity proxy measures. The utilized statistical measures and tests are commonly utilized to measure what we intended to measure, and the utilization of these measures and tests is in accordance with the empirical standards for software engineering research [42, 73].

7.3. Reliability. The reliability of a study is concerned with the reproducibility of a study and its findings [69, 74]. One potential threat that might have an impact on the reliability of this study is the reliance on human judgment to name the set of topics identified by LDA, as the reliance on human judgment may introduce some biases. To mitigate any potential introduced biases, both authors partake in the naming by reading the top 10 keywords and reviewing questions that have the highest scored probabilities of each topic. The naming was followed by a review step in which we reviewed a randomly selected sample of each identified topic to ensure the suitability of the assigned name of each topic. Moreover, eliminating biases that result from the reliance on humans to name topics could not be avoided, as LDA only clusters words based on their cooccurrence and similarity without providing names, and there is no reliable automated approach to achieve such a goal [16, 17].

8. Related Work

Several studies leveraged SO to understand real-world challenges discussed by the website users through the utilization of LDA. This section discusses related studies in the context of web development. In addition, the section includes studies that are not focused on web development, as they served as the foundation for the design of our study. To the best of our knowledge, no previous study has been conducted to address the Flask web framework's stance on SO.

Bajaj, Pattabiraman, and Mesbah [15] conducted an empirical analysis to understand what web developers discuss on SO. The authors retrieved any questions that contained "JavaScript," "CSS," or "HTML5" in one of their tags, resulting in over 500,000 SO questions. Through the utilization of LDA and a defined ranking algorithm, the authors found web development questions are increasing on the website. They also noted that though the number of browser-related questions is high, the share is decreasing over time. The authors also observed that form validation and other Document Object Model (DOM) questions have always been present on web developers discussions and that web developers are struggling with new HTML5 features, such as Canvas. The study also found that the topic of file handling to be one of the most popular topics, based on the number of views the topic received.

Venkatesh et al. [30] conducted an empirical study to understand what topics and trends client developers discussed about APIs on SO and the developers' forums of each included API. The study included 32 popular web APIs from seven different domains, such as YouTube as a video/audio streaming domain. The authors identified and retrieved web APIs-related questions on SO by searching for any question that has any of the API-related tags, such as the "youtube-api" tag, which retrieves questions related to the YouTube web API on SO. In addition, the authors developed a Python crawler to extract web APIs-related questions from its developer forums. These steps resulted in a dataset that contains 92,471 questions. The set of questions was analyzed using LDA, which revealed that APIs questions revolve around 40 topics. The authors furthered their analysis to unveil patterns within the questions and found that on average half of the questions related to API revolve around five dominant topics. Moreover, the authors observed that though the majority of topics are occasional questions, a small number of topics are persistent over time.

Mahmood et al. [29] conducted an empirical study to understand how web developers utilize web services. Through a tag-based filtering that includes a set of 54 web-services-related tags, the authors retrieved 233,785 web-services-related questions on SO. The utilization of LDA on the set of identified questions revealed that web-services questions revolve around 36 topics. The authors found that questions that are related to web-services deployment, architecture, and integration issues of applications to be the most popular topics and that the topic of customer identification using asp.net framework to be the most difficult topic. Moreover, the study found that the majority of the identified questions (i.e., 42%) were asked using "what."

Scoccia, Migliarini, and Autili [31] conducted an empirical study to understand what challenges developers face when they utilize desktop web application frameworks in general, without a specific focus on a particular framework. Through a tag-based filtering that includes nine tags on SO

questions and the collection of GitHub issues on framework application repositories, the authors acquired a dataset of 10,822 SO questions and 453 issue reports from GitHub applications. The utilization of LDA revealed that discussions related to desktop web application development revolve around 13 topics: app architecture, build and deploy, client-server, databases, dependencies, errors, file manipulation, interprocess communication, developer tools, page contents, platform integration, testing, and user interface. The study also found that the topics of user interface, build and deploy, and platform integration to be the most difficult topics, and that some topics are common on SO questions and on GitHub issues. Similarly, our study found that the deployment topic can be considered the second most difficult Flask-related topic.

Barua, Thomas, and Hassan [23] conducted a study that aims to understand topics of discussions and trends among SO users. Through the utilization of LDA, the authors concluded that developer discussions can be categorized into 40 topics. Eight of these topics are also present in our study's findings, which are background tasks, deployment, testing and logging, client–server, file handling, authentication and authorization, database, and best practices. Moreover, the study investigated trends within these identified topics and found that questions in some topics can lead to answers in other ones and that the most popular topics are web development, mobile development, Git, and MySQL.

Multiple other studies have utilized SO and LDA to gain a better understating of a number of software engineering subjects, such as refactoring [25], Docker development [19], Internet of things [24], mobile development [17], software ethics [21], software security [35], Firebase [36], chatbot development [20], new programing languages [37], big data [75], concurrency topics [18], modern release engineering [76], microservices [77], Android testing [78], Flutter [26], React Native [27] and more. Moreover, Silva, Galster, and Gilson [16] conducted a literature survey on topic modeling in software engineering research in general without focusing on SO. The survey thoroughly analyzed the majority of the studies summarized in this section. Hence, we recommend the reader to review the survey to gain a more comprehensive knowledge on topic modeling in the field of software engineering.

9. Conclusion

Flask is a popular web development framework that aims to facilitate web application development. It has gained widespread adoption among web developers since its initial release. This study aims to gain a better understanding of Flask through the lens of SO by presenting an analysis of SO Flask-related questions.

The study extracted and analyzed 70,230 Flask-related questions from SO. The study found that the interest towards Flask was generally increasing on SO until it peaked in 2020 to drop in 2021 and 2022. The utilization of LDA revealed that Flask-related questions revolve around 12 topics, which are as follows, based on decreasing frequency: database, form, deployment, file handling, setup, passing variables, client-server, background tasks, API, authentication and authorization, testing and logging, and best practices. Moreover, the utilization of proxy measures revealed that the identified topics vary in terms of their popularity and difficulty levels, where the topic of API can be considered the most popular topic, and the topic of background tasks can be considered the most difficult topic.

This study aims to gain an understanding of the stance of Flask on SO. The study provides individuals with a better insight on Flask and highlights and characterizes the hurdles that the framework users are facing. Software engineering researchers, practitioners, educators, and Flask contributors can leverage this study and its findings to guide their future efforts that are focused on Flask.

Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author, Luluh Albesher.

Conflicts of Interest

The authors declare no conflicts of interest.

Author Contributions

Luluh Albesher and Reem Alfayez contributed equally to this work.

Funding

The authors would like to thank the Deanship of Scientific Research (DSR) at King Saud University for funding and supporting this research through the initiative of DSR Graduate Students Research Support (GSR).

Acknowledgments

The authors would like to thank the Deanship of Scientific Research (DSR) at King Saud University for funding and supporting this research through the initiative of DSR Graduate Students Research Support (GSR).

References

[1] M. R. Mufid, A. Basofi, M. U. H. Al Rasyid, I. F. Rochimansyah, and A. rokhim, "Design an MVC Model Using Python for Flask Framework Development," in *2019 International Electronics Symposium (IES)*, (IEEE, 2019): 214–219.

[2] M. Jazayeri, "Some Trends in Web Application Development," in *Future of Software Engineering (FOSE'07)*, (IEEE, 2007): 199–213.

[3] D. Kaur and P. Kaur, "Empirical Analysis of Web Attacks," *Procedia Computer Science* 78 (2016): 298–306.

[4] S. Murugesan, "Web Application Development: Challenges and the Role of Web Engineering," in *Web Engineering: Modelling and Implementing Web Applications*, (Springer, 2008): 7–32.

[5] D. I. Samudio and T. D. LaToza, "Barriers in Front-End Web Development," in *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, (IEEE, 2022): 1–11.

[6] M. Asif, *Python for Geeks: Build Production-Ready Applications Using Advanced Python Concepts and Industry Best Practices* (Packt Publishing, 2021).
[7] S. Uzayr, *Mastering Python for Web: A Beginner's Guide*, Mastering Computer Science (CRC Press, 2022).
[8] "Flask Documentation," 2023, https://palletsprojects.com/p/flask/.
[9] M. Copperwaite and C. Leifer, *Learning Flask Framework"* (Packt Publishing Ltd., 2015).
[10] M. Grinberg, *Flask Web Development: Developing Web Applications With Python"* (O'Reilly Media, Inc, 2018).
[11] J. Stouffer, *Mastering Flask* (Packt Publishing, 2015).
[12] "Stack Overflow Survey," 2023, https://survey.stackoverflow.co/2022/.
[13] "Python Developers Survey," 2023, https://lp.jetbrains.com/python-developers-survey-2021/.
[14] "Stack Overflow Website," 2023, https://stackoverflow.com/.
[15] K. Bajaj, K. Pattabiraman, and A. Mesbah, "Mining Questions Asked by Web Developers," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, (Association for Computing Machinery, 2014): 112–121.
[16] C. C. Silva, M. Galster, and F. Gilson, "Topic Modeling in Software Engineering Research," *Empirical Software Engineering* 26, no. 6 (2021): 120.
[17] C. Rosen and E. Shihab, "What Are Mobile Developers Asking About? A Large Scale Study Using Stack Overflow," *Empirical Software Engineering* 21, no. 3 (2016): 1192–1223.
[18] S. Ahmed and M. Bagherzadeh, "What Do Concurrency Developers Ask About? A Large-Scale Study Using Stack Overflow," in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, (IEEE, 2018): 1–10.
[19] M. U. Haque, L. H. Iwaya, and M. A. Babar, "Challenges in Docker Development: A Large-Scale Study Using Stack Overflow," in *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, (IEEE, 2020): 1–11.
[20] A. Abdellatif, D. Costa, K. Badran, R. Abdalkareem, and E. Shihab, "Challenges in Chatbot Development: A Study of Stack Overflow Posts," in *Proceedings of the 17th International Conference on Mining Software Repositories*, (Association for Computing Machinery, 2020): 174–185.
[21] R. Alfayez, Y. Ding, R. Winn, and G. Alfayez, "What is Discussed About Software Engineering Ethics on Stack Exchange (q&a) Websites? A Case Study," in *2022 IEEE/ACIS 20th International Conference on Software Engineering Research, Management and Applications (SERA)*, (IEEE, 2022): 106–111.
[22] R. Alfayez, Y. Ding, R. Winn, G. Alfayez, C. Harman, and B. Boehm, "What Is Asked About Technical Debt (td) on Stack Exchange Question-and-Answer (q&a) Websites? An Observational Study," *Empirical Software Engineering* 28, no. 2 (2023): 35.
[23] A. Barua, S. W. Thomas, and A. E. Hassan, "What Are Developers Talking About? An Analysis of Topics and Trends in Stack Overflow," *Empirical Software Engineering* 19, no. 3 (2014): 619–654.
[24] G. Uddin, F. Sabir, Y.-G. Guéhéneuc, O. Alam, and F. Khomh, "An Empirical Study of Iot Topics in Iot Developer Discussions on Stack Overflow," *Empirical Software Engineering* 26, no. 6 (2021): 1–45.
[25] A. Peruma, S. Simmons, E. A. AlOmar, C. D. Newman, M. W. Mkaouer, and A. Ouni, "How Do i Refactor This? an Empirical Study on Refactoring Trends and Topics in Stack Overflow," *Empirical Software Engineering* 27, no. 1 (2022): 11.
[26] A. Alanazi and R. Alfayez, "What Is Discussed About Flutter on Stack Overflow (so) Question-and-Answer (q&a) Website: An Empirical Study," *Journal of Systems and Software* 215 (2024): 112089.
[27] L. Albesher, R. Aldossari, and R. Alfayez, "An Observational Study on React Native (rn) Questions on Stack Overflow (so)," *IET Software* 2023, no. 1 (2023): 6613434, 13 pages.
[28] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet Allocation," *Journal of Machine Learning Research* 3, no. Jan (2003): 993–1022.
[29] K. Mahmood, G. Rasool, F. Sabir, and A. Athar, "An Empirical Study of Web Services Topics in Web Developer Discussions on Stack Overflow," *IEEE Access* 11 (2023): 9627–9655.
[30] P. K. Venkatesh, S. Wang, F. Zhang, Y. Zou, and A. E. Hassan, "What Do Client Developers Concern When Using Web Apis? An Empirical Study on Developer Forums and Stack Overflow," in *2016 IEEE International Conference on Web Services (ICWS)*, (IEEE, 2016): 131–138.
[31] G. L. Scoccia, P. Migliarini, and M. Autili, "Challenges in Developing Desktop Web Apps: A Study of Stack Overflow and Github," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, (IEEE, 2021): 271–282.
[32] E. Peregrino, *Programming Raspberry Pi in 30 Days: Learn How to Build Amazing Raspberry Pi Projects Using Python With Ease (English Edition)* (BPB Publications, 2023).
[33] "Stack Exchange Network Website," 2023, https://stackexchange.com/sites.
[34] H. Jelodar, Y. Wang, C. Yuan, et al., "Latent Dirichlet Allocation (LDA) and Topic Modeling: Models, Applications, a Survey," *Multimedia Tools and Applications* 78, no. 11 (2019): 15169–15211.
[35] X.-L. Yang, D. Lo, X. Xia, Z.-Y. Wan, and J.-L. Sun, "What Security Questions Do Developers Ask? a Large-Scale Study of Stack Overflow Posts," *Journal of Computer Science and Technology* 31, no. 5 (2016): 910–924.
[36] M. S. A. Khan, A. R. Farabi, and A. Iqbal, "What Do Firebase Developers Discuss About? An Empirical Study on Stack Overflow Posts," in *Proceedings of the 9th International Conference on Networking, Systems and Security*, (Association for Computing Machinery, 2022): 63–74.
[37] P. Chakraborty, R. Shahriyar, A. Iqbal, and G. Uddin, "How Do Developers Discuss and Support New Programming Languages in Technical q&a Site? An Empirical Study of Go, Swift, and Rust in Stack Overflow," *Information and Software Technology* 137 (2021): 106603.
[38] D. S. Cruzes and T. Dyba, "Recommended Steps for Thematic Synthesis in Software Engineering," in *2011 International Symposium on Empirical Software Engineering and Measurement*, (IEEE, 2011): 275–284.
[39] S. Fincher and J. Tenenberg, "Making Sense of Card Sorting Data," *Expert Systems* 22, no. 3 (2005): 89–93.
[40] P. Vogel, T. Klooster, V. Andrikopoulos, and M. Lungu, "A Low-Effort Analytics Platform for Visualizing Evolving Flask-Based Python Web Services," in *2017 IEEE Working Conference on Software Visualization (VISSOFT)*, (IEEE, 2017): 109–113.
[41] "Stack Exchange Data Dump," 2023, https://archive.org/details/stackexchange.
[42] P. Ralph, N. b. Ali, S. Baltes, et al., "Empirical Standards for Software Engineering Research," arXiv preprint arXiv: 2010.03525 (2020).

[43] J. R. Landis and G. G. Koch, "The Measurement of Observer Agreement for Categorical Data," in *Biometrics*, (1977): 159–174.
[44] "Nltk Documentation," 2023 https://www.nltk.org/.
[45] "Spacy Lemmatizer Documentation," 2023, https://spacy.io/api/lemmatizer/.
[46] "Mallet Documentation," 2023, https://mimno.github.io/Mallet/topics.
[47] "Flask-Sqlalchemy Documentation," 2023, https://pypi.org/project/Flask-SQLAlchemy/.
[48] "Flask-Migrate Documentation," 2023, https://pypi.org/project/Flask-Migrate/.
[49] "Flask Wtform Documentation," 2023, https://pypi.org/project/Flask-WTF/.
[50] K. Relan and K. Relan, "Deploying Flask Applications," in *Building REST APIs With Flask: Create Python Web Services With MySQL*, (2019): 159–182.
[51] "Apache," 2023, https://httpd.apache.org/.
[52] "Nginx," 2023, https://nginx.org/.
[53] "Heroku," 2023, https://www.heroku.com/.
[54] "Aws,", accessed: 2023-11-08 https://aws.amazon.com/.
[55] E. Rosebrock, "Creating Interactive Websites With PHP and Web Services," (Wiley (2006).
[56] P. Consulting, *Data Science for Business Professionals: A Practical Guide for Beginners (English Edition)* (BPB PUBN, 2020).
[57] T. Ziade, *Python Microservices Development: Build, Test, Deploy, and Scale Microservices in Python* (Packt Publishing, 2017).
[58] "Celery Documentation," 2023, https://docs.celeryq.dev/en/stable/.
[59] "Redis," 2023, https://redis.com/solutions/use-cases/messaging/.
[60] "Rabbitmq," 2023, https://www.rabbitmq.com/.
[61] M. Lathkar, *Building Web Apps With Python and Flask: Learn to Develop and Deploy Responsive RESTful Web Applications Using Flask Framework (English Edition)* (BPB PUBN).
[62] D. Gaspar and J. Stouffer, "Mastering Flask Web Development: Build Enterprise-Grade, Scalable Python Web Applications," (Packt Publishing (2018).
[63] "Flask-Login Documentation," 2023, https://pypi.org/project/Flask-Login/.
[64] L. Mamykina, B. Manoim, M. Mittal, G. Hripcsak, and B. Hartmann, "Design Lessons From the Fastest q&a Site in the West," in *Proceedings of the SIGCHI Conference on Human Factors in computing systems*, (Association for Computing Machinery, 2011): 2857–2866.
[65] M.-T. Puth, M. Neuh'auser, and G. D. Ruxton, "Effective Use of Spearman's and Kendall's Correlation Coefficients for Association Between Two Measured Traits," *Animal Behaviour* 102 (2015): 77–84.
[66] P. Schober, C. Boer, and L. A. Schwarte, "Correlation Coefficients: Appropriate Use and Interpretation," *Anesthesia & Analgesia* 126, no. 5 (2018): 1763–1768.
[67] M. Hubert and P. Rousseeuw, *International Encyclopedia of Statistical Science"* (Springer-Verlag, 2010).
[68] J. Han, E. Shihab, Z. Wan, S. Deng, and X. Xia, "What Do Programmers Discuss About Deep Learning Frameworks," *Empirical Software Engineering* 25, no. 4 (2020): 2694–2747.
[69] R. K. Yin, "Case Study Research: Design and Methods," *Sage* 5 (2009).
[70] B. Kitchenham, D. Budgen, and P. Brereton, *Evidence-Based Software Engineering and Systematic Reviews*, Chapman & Hall/CRC Innovations in Software Engineering and Software Development Series. (CRC Press, 2015).
[71] C. Wohlin, P. Runeson, M. H'ost, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, Computer Science (Springer, Berlin Heidelberg, 2012).
[72] R. Alfayez, R. Winn, Y. Ding, G. Alfayez, and B. Boehm, "Technical Debt (td) Through the Lens of Twitter: A Survey," *Journal of Software: Evolution and Process* (2023): e2547.
[73] N. Gisev, J. S. Bell, and T. F. Chen, "Interrater Agreement and Interrater Reliability: Key Concepts, Approaches, and Applications," *Research in Social and Administrative Pharmacy* 9, no. 3 (2013): 330–338.
[74] P. Runeson, M. Host, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples* (John Wiley & Sons, 2012).
[75] M. Bagherzadeh and R. Khatchadourian, "Going Big: A Large-Scale Study on What Big Data Developers Ask," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, (Association for Computing Machinery, 2019): 432–442.
[76] M. Openja, B. Adams, and F. Khomh, "Analysis of Modern Release Engineering Topics—A Large-Scale Study Using Stackoverflow," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, (IEEE, 2020): 104–114.
[77] A. Bandeira, C. A. Medeiros, M. Paixao, and P. H. Maia, "We Need to Talk About Microservices: An Analysis From the Discussions on Stackoverflow," in *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, (IEEE, 2019): 255–259.
[78] I. K. Villanes, S. M. Ascate, J. Gomes, and A. C. Dias-Neto, "What are Software Engineers Asking About Android Testing on Stack Overflow?" in *Proceedings of the XXXI Brazilian Symposium on Software Engineering*, (IEEE, 2017): 104–113.